

1. Хранение информации

Архитектура компьютера – структурная организация компьютера в виде совокупности функциональных модулей иопределенных связей между ними.

Хранение информации — это процесс фиксации информации на материальном носителе с целью её сохранения во времени и последующего извлечения.

Информация — это абстрактное понятие, которое нельзя увидеть или потрогать. Однако информация не может существовать без материального носителя.

• Носители информации

Информация не может существовать без материального носителя. Носитель — это средство переноса информации в пространстве и во времени. Носителем может быть:

- Физический объект (бумага, магнитный диск)
- Энергетический субстрат (электрические, световые, звуковые сигналы)

• Кодирование при хранении

При отображении информации на носителе она кодируется — ей ставится в соответствие форма, цвет или структура сигналов. От выбора носителя и способа кодирования зависит эффективность работы системы.

В компьютере информация кодируется в двоичной системе счисления. Наименьшая единица хранения — бит (0 или 1). В современных компьютерах также используется байт = 8 бит.

• Память компьютера

От памяти ЭВМ требуется три способности:

- Выбрать определённую информацию
- Сохранить её
- Извлечь по требованию

• Физическое представление битов

В ЭВМ 0 и 1 представляются электрическими сигналами.

Два основных способа:

Способ	1	0
Импульсный	Есть импульс	Нет импульса
Потенциальный	Высокое напряжение	Низкое напряжение

При потенциальном способе уровень напряжения не меняется в течение такта работы машины.

Заключение: Хранение информации в ЭВМ — это представление данных в двоичном виде (биты) и фиксация их на материальном носителе с помощью электрических сигналов. Биты объединяются в байты, регистры и ячейки памяти, которые составляют память компьютера.

2. Адекватность информации: синтаксическая, семантическая, прагматическая

❖ Что такое адекватность информации

Адекватность информации — это соответствие содержания информации образу отображаемого объекта. В свете идей семиотики (науки о знаковых системах) адекватность выражается в трёх формах: синтаксической, семантической и прагматической.

1. Синтаксическая адекватность

Связана с воспроизведением формально-структурных характеристик отражения, абстрагируясь от смысловых и потребительских параметров.

На синтаксическом уровне учитываются:

- Тип носителя
- Способ представления информации
- Скорость передачи и обработки
- Формат кодов представления
- Надёжность и точность преобразования

Информацию, рассматриваемую только с синтаксических позиций, называют данными.

2. Семантическая адекватность

Выражает аспект соответствия образа, знака и объекта — то есть отношение информации и её источника.

Семантическая информация проявляется только при наличии единства информации и пользователя. На этом уровне анализируется смысловое содержание информации, смысловые связи между кодами представления. Пример: Если я вижу лист бумаги с текстом, то с семантической точки зрения я понимаю, о чём этот текст, какой смысл он несёт.

3. Прагматическая адекватность

Отражает отношение информации и её потребителя, соответствие информации и цели управления. Прагматические свойства проявляются только при наличии трёх элементов:

- Информация (объект)
- Пользователь (субъект)
- Цель управления

Прагматический аспект связан с ценностью информации для выработки управленческого решения. Пример: Текст «2×2=4» для студента, который это уже знает, имеет низкую прагматическую ценность. А новая информация, помогающая решить задачу, — высокую.

▪ Связь с тремя ступенями познания истины

Ступень познания	Соответствует
Живое созерцание	Синтаксической стороне (восприятие внешних характеристик)
Абстрактное мышление	Семантической стороне (формирование понятий, выявление смысла)
Практика	Прагматической стороне (использование информации для деятельности)

Заключение:Таким образом, одна и та же информация может рассматриваться на трёх уровнях:

Синтаксис — как данные (объём, формат, носитель)

Семантика — как смысл (что означает)

Прагматика — как ценность (помогает ли достичь цели)

3. Меры информации

1. **Общий подход** В соответствии с тремя формами адекватности существуют разные способы измерения информации. Терминологически различают объём данных (синтаксический уровень) и количество информации (семантический и прагматический уровни).

2. Синтаксические меры информации (объём данных)

Объём данных в сообщении измеряется количеством символов (разрядов) принятого алфавита.

Число возможных состояний, которые можно отобразить: $N = m^n$ где: N — число всевозможных отображаемых состояний

m — основание системы счисления

n — число разрядов (символов) в сообщении

Единицы измерения:

- В двоичной системе: бит (binary digit)
- В десятичной системе: дит (decimal digit)
- В современных компьютерах: байт = 8 бит

Количество информации

Определение количества информации на синтаксическом уровне невозможно без понятия неопределённости состояния (энтропии) системы.

Получение информации связано с изменением степени неосведомлённости получателя. Количество информации измеряется изменением неопределённости состояния системы. Если конечная неопределённость обращается в нуль, то неполное знание заменяется полным.

3. Семантическая мера информации (тезаурусная мера Шнейдера)

Синтаксические меры не могут измерить смысловое содержание информации. Для этого используется тезаурусная мера, предложенная Ю.И. Шнейдером.

Тезаурус — совокупность сведений, которыми располагает пользователь.

Принцип:

- При $S_n = 0$ — пользователь не воспринимает информацию $\rightarrow I_c = 0$;
- при $S_n \rightarrow \infty$ — пользователь всё знает, информация не нужна $\rightarrow I_c = 0$;
- **максимум I_c** — когда информация понятна пользователю и несёт ранее неизвестные сведения

Прагматическая мера информации: Это полезность информации, её ценность для пользователя (управления). Эта мера относительна и обусловлена особенностями использования информации в конкретной системе управления.

Ценность информации целесообразно измерять в тех же единицах, что и целевая функция управления системой. **Заключение**

Уровень	Что измеряется	Мера
Синтаксический	Объём данных	Бит, байт, дит, $N=m^n$
Семантический	Смысловое содержание	Тезаурусная мера Шнейдера

Прагматический	Ценность, полезность	Зависит от целевой функции управления
----------------	----------------------	---------------------------------------

4. Архитектура компьютера

Архитектура компьютера — это структурная организация компьютера в виде совокупности функциональных модулей и определенных связей между ними.

Архитектура определяется совокупностью свойств, существенных для пользователя. Например, разрядность компьютера или его быстродействие — это часть архитектуры. А марка вентилятора или цвет корпуса — не относятся к архитектуре.

Основное внимание при этом уделяется структуре и функциональным возможностям машины.

➤ Функции компьютера

Функции компьютера делятся на основные и дополнительные.

Основные функции

Они определяют назначение ЭВМ:

- Обработка информации
- Хранение информации
- Обмен информацией с внешними объектами

Это то, что было заложено в самых первых компьютерах.

Дополнительные функции

Они повышают эффективность выполнения основных функций:

- Обеспечение эффективных режимов работы
- Диалог с пользователем (работа в форме меню, не надо запоминать команды)
- Высокая надежность

➤ Реализация функций

Функции ЭВМ реализуются с помощью её компонентов:

- **Аппаратные средства** (процессор, память, устройства ввода-вывода)
- **Программные средства** (операционная система, прикладные программы)

Часть функций реализуется операционной системой.

➤ Архитектура и многоуровневая организация

Компьютер можно представить как многоуровневую структуру (из второй лекции). Каждый уровень — это виртуальная машина, которая выполняет свои функции.

Уровень	Название	Что делает
0	Цифровой логический уровень	Вентили, биты памяти, регистры
1	Микроархитектура	Регистры, АЛУ, тракт данных
2	Набор команд (ассемблер)	Машинные команды
3	Операционная система	Многозадачность, управление памятью
4	Языки высокого уровня	C++, Python, Java
5	Прикладная предметная область	Задачи пользователя

Архитектура компьютера — это набор типов данных, операций и характеристик каждого уровня. Благодаря иерархической структуре мы можем не обращать внимания на детали нижних уровней и работать на том уровне, который нам нужен.

- **Целостность и структура системы**
Важные понятия, связанные с архитектурой:
Система — совокупность элементов, взаимодействующих друг с другом, образующих целостность и единство.
Элемент системы — часть системы, имеющая определенное функциональное назначение.
Подсистема — сложный элемент, состоящий из более простых взаимосвязанных элементов.
Организация системы — внутренняя упорядоченность, согласованность взаимодействия элементов.
Структура системы — состав, порядок и принципы взаимодействия элементов, определяющие основные свойства системы.
Целостность системы (эмерджентность) — свойства системы не сводятся к сумме свойств её отдельных элементов, и в то же время свойства каждого элемента зависят от его места и функции внутри системы.
- **Иерархическая структура**
Если элементы системы разнесены по разным уровням и внутренние связи организованы только от вышестоящих к нижестоящим уровням и наоборот, то говорят об иерархической структуре системы. Компьютер как раз имеет такую структуру.

5. Концепция хранимой программы

Проблема: калькулятор бесконечно ожидает каждую команду с клавиатуры. В ЭВМ потери времени между командами устраняются тем, что сами команды хранятся в ОП → команды тоже должны быть последовательностью бит.

Концепция хранимой программы — идея использования памяти для хранения и команд, и данных. Впервые высказана и внедрена **Джоном фон Нейманом в 1940 году**. Заслуга: команды можно интерпретировать как данные и выполнять над ними арифметические операции; стало возможно быстро менять порядок выполнения с помощью условного и безусловного перехода. В 1946 году Д. фон Нейман, Г. Голдстейн, А. Беркс изложили новые принципы построения ЭВМ.

Принципы фон Неймана:

- Использование двоичной системы счисления.
- Программное управление ЭВМ.
- Память используется для хранения и данных, и программ.
- Ячейки памяти имеют последовательно пронумерованные адреса.
- Возможность условного перехода в процессе выполнения программы.

6. Архитектура адресации операндов для работы процессора

- Для работы процессора команда должна содержать:
- Код операции (КОП) — что делать
 - Адреса операндов — откуда взять данные
 - Адрес следующей инструкции (в ранних машинах)

➤ Эволюция адресации (от 3+1 к безадресной)

Тип машины	Структура команды	Пример $c = a + b$
3+1 адресная	КОП + 3 адреса операндов + адрес следующей команды	ADD a, b, c, next
Трёхадресная	КОП + 3 адреса (команды расположены подряд, адрес следующей не нужен)	ADD a, b, c
Двухадресная	КОП + 2 адреса (результат на место одного операнда)	MOV c, a → ADD c, b
Одноадресная	КОП + 1 адрес (второй операнд — аккумулятор)	LAC a → ADD b → DAC c
Безадресная	Только КОП (операнды на стеке)	PUSH a → PUSH b → ADD → POP c

- **Почему перешли к двухадресной**
 - Экономия памяти — команда становится короче на один адрес
 - Экономия времени — меньше данных нужно выбирать из памяти
 - Результат помещается на место одного из операндов (обычно первого)
- **Одноадресная машина**
 - Нужен аккумулятор (специальный регистр) для хранения второго операнда
 - Команды: LAC (загрузить в аккумулятор), DAC (сохранить из аккумулятора)
- **Безадресная машина**
 - Испол-ся стек — операнды неявно берутся из стека
 - PUSH — положить в стек, POP — взять из стека
 - Арифметические команды (ADD, MUL, SUB) не имеют операндов — берут два верхних элемента стека, результат кладут обратно в стек

Сравнение $a = b * c - d * e$

3-адресная	2-адресная	1-адресная	безадресная
MUL b,c,a	MOV a,b	LAC d	PUSH d
MUL d,e,t	MUL a,c	MUL e	PUSH e
SUB a,t,a	MOV t,d	DAC a	MUL
	MUL t,e	LAC b	PUSH b
	SUB a,t	MUL c	PUSH c
		SUB a	MUL
		DAC a	SUB
			POP a

7. Работа ЦП.

Центральный процессор — это «мозг» компьютера. Его задача — выполнять программы, находящиеся в основной памяти. Он вызывает команды из памяти, определяет их тип и выполняет одну за другой.

➤ Состав ЦП

Компонент	Функция
Блок управления (УУ)	Отвечает за вызов команд из памяти и определение их типа
Арифметико-логическое устройство (АЛУ)	Выполняет арифметические (сложение, умножение) и логические (И, ИЛИ) операции
Регистровая память	Хранит промежуточные результаты и некоторые команды управления

➤ Основные регистры ЦП

Регистр	Функция
Счетчик команд (PC)	Хранит адрес следующей команды
Регистр команд (IR)	Хранит текущую выполняемую команду
Регистр адреса (MAR)	Хранит адрес ячейки памяти для чтения/записи
Регистр данных (MBR/MDR)	Хранит данные, прочитанные из памяти или предназначенные для записи

➤ Цикл работы ЦП

Процессор выполняет каждую команду за несколько шагов:

Шаг	Действие
1	Вызывает следующую команду из памяти и переносит её в регистр команд
2	Увеличивает счетчик команд, чтобы он указывал на следующую команду
3	Определяет тип вызванной команды (декодирование)
4	Если команда использует слово из памяти, определяет его адрес
5	Переносит слово (если нужно) в регистр ЦП
6	Выполняет команду
7	Переходит к шагу 1

Эта последовательность является основой работы всех компьютеров независимо от их архитектуры.

➤ Тракт данных

Тракт данных — это часть ЦП, состоящая из АЛУ, его входов и выходов, а также регистров и шин, их соединяющих.

Как работает тракт данных:

- Выбираются один или два регистра с операндами
- Операнды поступают на вход АЛУ
- АЛУ выполняет операцию
- Результат помещается в один из регистров или в память

➤ Типы команд по работе с памятью

Тип команд	Описание
Регистр-память	Вызывают слова из памяти, помещают в регистры, затем АЛУ работает с регистрами
Регистр-регистр	Вызывают два операнда из регистров, выполняют операцию, результат возвращают в регистр

➤ Организация связи с памятью и устройствами

ЦП, память и интерфейсы устройств соединяются шиной, которая обеспечивает три вида передачи:

- ЦП ↔ ОП
- ЦП ↔ интерфейс устройства
- ОП ↔ интерфейс устройства

Шина включает:

- Шина адреса — передаёт адрес
- Шина данных — передаёт данные
- Шина управления — передаёт сигналы (чтение/запись)

8. Интерпретатор команды или сложная команда

Проблема: как выполнить сложные команды на простом компьютере.

Разработчик, выбрав машинный язык для нового компьютера, должен решить:

- Разработать процессор, который аппаратно выполняет все команды (дорого)
- Или написать интерпретатор — программу, которая имитирует работу процессора (дешевле)

Интерпретатор — это программа, которая:

- Написана на машинном языке более простого компьютера
- Принимает на вход команды целевого машинного языка
- Разбивает каждую сложную команду на более мелкие, элементарные
- Выполняет их последовательно

Интерпретатор можно написать так, что он будет выполнять любую систему команд, даже очень сложную.

Зачем нужна интерпретация: Совместимость, Экономия стоимости, Семейство компьютеров, Гибкость

➤ Гибридный подход

Возможны гибридные конструкции, когда:

- Часть команд выполняется аппаратно (простые, частые)
- Часть команд выполняется интерпретатором (сложные, редкие)

RISC и CISC (1980, Д. Паттерсон и К. Секвин, Беркли):

-RISC (Reduced Instruction Set Computer) — сокращённый набор команд, ориентация не на интерпретацию;

- CISC (Complex Instruction Set Computer) — полный набор команд

Достоинства и недостатки интерпретации

Достоинства	Недостатки
Дешевле, чем сложный процессор	Медленнее аппаратного выполнения
Позволяет создавать семейства совместимых машин	Требует дополнительной памяти для интерпретатора
Упрощает разработку новых архитектур	Интерпретатор сам занимает время процессора

9. Параллелизм на уровне команд

Параллелизм на уровне команд (ILP — Instruction-Level Parallelism) — это способность процессора выполнять несколько команд одновременно (параллельно) или с перекрытием во времени.

Существуют две формы параллелизма:

Форма	Описание
Параллелизм на уровне команд	Запуск большого количества команд каждую секунду (конвейеры, суперскалярность)
Параллелизм на уровне процессоров	Несколько процессоров работают над одним заданием (мультипроцессоры)

Три основных подхода к увеличению быстродействия

Подход	Суть
Сокращение количества циклов	Уменьшить число тактов на выполнение команды (RISC — 1 такт)
Упрощение организации	Сделать цикл короче (повысить тактовую частоту)
Одновременное выполнение	Выполнять несколько операций параллельно (конвейеры, суперскалярность)

На практике чаще всего используются все три подхода вместе.

Конвейеризация (основной способ)

Конвейер — это техника, при которой выполнение команды разбивается на несколько независимых этапов (ступеней), каждый из которых обрабатывается своим аппаратным блоком. Пока одна команда выполняет один этап, другая команда — следующий, и т.д.

Пример 4-ступенчатого конвейера (Intel 80286):

- Адресный блок — вычисление адреса команды
- Блок выборки — чтение команды из памяти
- Дешифрация — определение типа команды
- Выполнение — исполнение команды

5-ступенчатый конвейер:

Ступень	Действие
IF	Выборка команды (Instruction Fetch)
ID	Дешифрация (Instruction Decode)
OR	Выборка операндов (Operand Read)
EX	Выполнение (Execute)
WB	Обратная запись результата (Write Back)

Конфликты конвейера

Конфликт — это ситуация, когда очередная команда не может выполняться в предназначенном для неё такте.

Тип конфликта	Причина	Пример
Структурные	Аппаратные средства не поддерживают одновременное выполнение	Команда и операнды в одной памяти
По управлению	Команды условного перехода	Неизвестно, какую ветку загружать
По данным	Зависимость команд по данным	$R1 = R2 + R3 \rightarrow R4 = R1 + R5$

Способы борьбы с конфликтами

Конфликт	Способы решения
Структурные	Увеличение такта, дублирование ресурсов, раздельные память команд и данных
По управлению	Динамическое прогнозирование ветвлений, статическое прогнозирование, спекулятивное выполнение
По данным	Форвардинг (прямая передача), приостановка конвейера

Суперскалярные архитектуры

Суперскалярный процессор — процессор с несколькими конвейерами, работающими параллельно.

- Один блок выборки команд обслуживает несколько конвейеров
- Команды группируются в пары (или больше) и выполняются одновременно
- Требуются специальные компиляторы, которые находят независимые команды для параллельного выполнения

Пример: процессор Pentium имел вдвоенный пятиступенчатый конвейер.

Длина конвейера

Длина конвейера	Преимущества	Недостатки
Короткий (4-5 ступеней)	Меньше потерь при конфликтах	Ниже тактовая частота
Длинный (20+ ступеней)	Выше тактовая частота	Большие потери при перезагрузке конвейера

В современных процессорах отчасти отказались от чрезмерно длинных конвейеров из-за высоких потерь при неверном прогнозировании переходов.

10. Параллелизм на уровне процессоров

Параллелизм на уровне процессоров — это использование нескольких процессоров для совместного выполнения одного задания. В отличие от параллелизма на уровне команд (конвейеры, суперскалярность), здесь работают независимые процессоры, а не отдельные блоки внутри одного.

- **Почему нужен параллелизм на уровне процессоров**
 - Конвейеры и суперскалярность повышают скорость в 5–10 раз
 - Чтобы увеличить производительность в 50–100 раз, нужны многопроцессорные системы
 - Ограничения: скорость света (20 см/нс), тепловыделение, энергопотребление
- **Классификация по потокам команд и данных (архитектура Флинна)**

Тип	Команды	Данные	Пример
SISD	Один поток	Один поток	Классический фон-Неймановский комп-ер
SIMD	Один поток	Много потоков	Матричные процессоры, векто-ые процессоры
MIMD	Много потоков	Много потоков	Мультипроцессоры, мультикомпьютеры

- **Матричные процессоры (SIMD)**
Принцип: Много одинаковых простых процессоров выполняют одну и ту же команду над разными наборами данных.
 - Есть один блок управления, который рассылает команды всем процессорам
 - Каждый процессор имеет свою локальную память
 - Процессоры расположены в виде матрицы (например, 8×8)
- **Векторные процессоры (SIMD)**
Отличие от матричных: не много процессоров, а один процессор с векторными регистрами и конвейерным АЛУ.
Принцип работы:
 - Одна команда загружает вектор (массив чисел) из памяти в векторный регистр
 - Одна команда выполняет операцию над всеми элементами вектора попарно
 - Результат — вектор, сохраняемый обратно в память

Особенности:

- Векторные регистры могут иметь разную длину
- Суммирующее устройство имеет конвейерную структуру
- Пионер — компания Cray Research

➤ **Мультипроцессоры (MIMD, общая память)**

Мультипроцессор — система из нескольких параллельных процессоров, имеющих общую память.

Характеристики:

- Каждый процессор может записывать и читать из любой части общей памяти
- Требуется синхронизация для безопасной работы с общей памятью
- Называются сильносвязанными

Проблемы:

- При большом числе процессоров (более 256) сложно обеспечить синхронизацию
- Конфликты при одновременном обращении к памяти

- **Мультикомпьютеры (MIMD, локальная память)**
Мультикомпьютер — система процессоров, каждый из которых имеет свою локальную память.

Характеристики:

- Общая память может отсутствовать или использоваться редко
- Процессоры общаются через сообщения (как в сети)
- Меньше проблем с синхронизацией

Причина появления:

- Мультипроцессоры с общей памятью сложно масштабировать
- Проще объединить процессоры с локальной памятью через сеть

➤ **Сравнение систем**

Тип	Память	Связь	Сложность синхронизац
Мультипроцессор	Общая	Шина	Высокая
Мультикомпьютер	Локальная	Сообщения	Низкая

11. Конвейер в ЦП.

Конвейер (pipeline) — это техника организации работы процессора, при которой выполнение команды разбивается на несколько последовательных этапов (ступеней). Каждый этап обрабатывается своим аппаратным блоком, и блоки работают параллельно над разными командами.

В то время как одна команда находится на этапе выполнения, следующая — на этапе дешифрации, а третья — на этапе выборки.

➤ **Почему конвейер повышает производительность**

Без конвейера процессор выполняет команды последовательно:

Выборка → (ожидание) → Дешифрация → (ожидание) → Выполнение → (ожидание) → ...

С конвейером этапы перекрываются:

Пока выполняется команда 1, выбирается команда 2, дешифрируется команда 3 и т.д.

- Важно: Конвейер не ускоряет выполнение одной отдельной команды. Он увеличивает пропускную способность — количество команд, выполняемых за единицу времени.

Время выполнения команды в конвейере:

$T = \max(T_1, T_2, T_3, T_4, T_5) + \Delta t$

(Определяется самой медленной ступенью + задержка на буферы между ступенями)

Сравнение для n команд:

Тип	Формула
Последовательное выполнение	$T_{\text{посл}} = n \times (T_1 + T_2 + T_3 + T_4 + T_5)$
Конвейер (5 ступеней)	$T_{\text{конв}} = 5T + (n-1) \times T$

Ускорение при большом n стремится к числу ступеней (в идеальном случае — к 5).

12. Суперскалярная архитектура

Суперскалярная архитектура — это архитектура процессора, которая имеет несколько конвейеров, работающих параллельно. За один такт такой процессор может выбирать, декодировать и выполнять несколько команд одновременно.

Процессор с одним конвейером — скалярный.
Процессор с двумя и более конвейерами — суперскалярный.

➤ **Принцип работы**

- Один блок выборки команд обслуживает все конвейеры
- Выбранные команды группируются в пары (или более) и распределяются по конвейерам
- Команды выполняются параллельно, каждая в своём конвейере

➤ **Условие эффективной работы**

Команды, выполняемые параллельно, должны быть независимыми:

- Не использовать результаты друг друга
- Не обращаться к одним и тем же ресурсам (регистрам, ячейкам памяти)

Если команды зависимы — они выполняются последовательно (одна за другой).

➤ **Роль компилятора**

Для эффективной работы суперскалярного процессора требуется поддержка компилятора:

Задача компилятора	Описание
Поиск независимых команд	Определение пар команд, которые можно выполнять параллельно
Переупорядочивание	Группировка независимых команд рядом друг с другом
Оптимизация	Учёт особенностей конвейеров процессора

Пример: процессор Pentium (один из первых массовых суперскалярных) требовал специальных компиляторов, которые объединяли совместимые команды в пары.

➤ **Сравнение: скалярный vs суперскалярный**

Характеристика	Скалярный	Суперскалярный
Количество конвейеров	1	2 и более
Команд за такт	1	2–6 (и более)
Сложность аппаратуры	Низкая	Высокая
Требования к компилятору	Минимальные	Специальная оптимизация

13. Память

Память — одна из главных частей компьютера. Без памяти машина работать не может, так как согласно концепции хранимой программы фон Неймана, в памяти хранятся и программа, и данные.

➤ **Характеристики систем памяти**

Характеристика	Описание
Местоположение	Процессорная (регистры, кэш L1), внутренняя (кэш L2/L3, ОЗУ), внешняя (диски, ленты)
Ёмкость	Количество битов или байтов (кило, мега, гига, тера — степени двойки)
Единица пересылки	Для ОЗУ — ширина шины данных (слово), для внешней — блок

Метод доступа	Последовательный, прямой, произвольный, ассоциативный
Быстродействие	Время доступа, длительность цикла, скорость передачи
Физический тип	Полупроводниковая, магнитная, оптическая
Стоимость	Влияет на выбор объёма и типа памяти

➤ **Иерархия памяти**

Уровень	Скорость	Ёмкость	Стоимость	Энергозависимость
Регистры	Самая быстрая	Очень малая	Высокая	Да
Кэш L1	Очень быстрая	16–64 КБ	Высокая	Да
Кэш L2/L3	Быстрая	256 КБ – несколько МБ	Средняя	Да
Оперативная память	Средняя	4–64 ГБ	Низкая	Да
Внешняя память (диск)	Медленная	Большая ТБ+)	Очень низкая	Нет

➤ **Оперативная память (ОЗУ)**

- Состоит из адресуемых ячеек (обычно 8 бит = 1 байт)
 - Каждая ячейка имеет уникальный адрес
 - Произвольный доступ — время доступа одинаково для всех ячеек
- Энергозависима — данные теряются при выключении

➤ **Кэш-память**

- Небольшой объём быстрой памяти, расположенный близко к процессору
- Основана на принципах временной и пространственной локальности
- Процессор работает с кэшем до 90% времени
- Бывает раздельная (команды и данные отдельно — Гарвардская архитектура) и объединённая

➤ **Внешняя память**

- Магнитные диски, ленты, оптические диски
- Энергонезависима — данные сохраняются
- Большая ёмкость, низкая скорость
- ЦП не может работать с ней напрямую — данные сначала копируются в ОЗУ

Почему нужна и ОЗУ, и внешняя память

Тип памяти	Почему нельзя обойтись
Только ОЗУ	Энергозависима (при выключении данные теряются), малый объём
Только внешняя	ЦП не может напрямую выполнять код с диска — только из ОЗУ

➤ **Заключение**

Память в компьютере — это иерархическая система от быстрых и дорогих регистров до медленных и дешёвых дисков. Ключевые идеи:

Адресация — каждая ячейка имеет номер

Кэширование — использование локальности для ускорения. **Компромисс** между скоростью, ёмкостью и стоимостью

14. Способы адресации.

Способ адресации — это метод, с помощью которого команда процессора определяет, где находятся операнды (данные) для выполнения операции.

➤ Размер адреса

Размер адреса определяет максимальный объём адресуемой памяти:

- 16 разрядов → 64 Кбайт
- 32 разряда → 4 Гбайт
- 64 разряда → 16 Эксбайт

➤ Классификация способов адресации

Способ	Суть	Пример (ассемблер)
Неявная	Адрес подразумевается по коду операции	STD (установить флаг)
Непосредственная	Операнд находится в самой команде	MOV AX, 1234h
Прямая	Адрес операнда прямо указан в команде	ADD AX, BX (регистровая)
Косвенная	В команде указан адрес адреса (регистр или выражение)	ADD AX, [BX]
Относительная	Адрес = текущий PC + смещение (для переходов)	JC 12h
Адресация ввода/вывода	Доступ к устройствам (отдельное или совмещённое пространство)	IN AL, 60h

➤ Виды косвенной адресации

Эффективный адрес (EA) вычисляется во время выполнения:

Подвид	Формула EA	Пример
Регистровая	EA = [R]	ADD AX, [BX]
Базовая	EA = [R] + смещение	MOV AX, [BX+100h]
Индексная	EA = [SI] + смещение (обычно SI меняется)	SUB AX, [SI+200h]
Базовая индексная	EA = [BX] + [SI] + смещение	MOV AX, [BX+SI+50h]

➤ Адресация ввода/вывода

Тип	Пространство	Команды	Пример архитектуры
Отдельное	Своё адресное пространство	Специальные (IN, OUT)	Intel x86
Совмещённое (MMIO)	Часть общего адресного пространства	Обычные (MOV)	Motorola, PowerPC

15. Управление памятью.

Управление памятью — это совокупность аппаратных и программных средств, обеспечивающих преобразование адресов, защиту памяти и реализацию виртуальной памяти. Ключевой момент — преобразование адреса: превращение адресов, используемых программами (логических), в адреса, воспринимаемые схемами памяти (физические).

➤ Реальный режим (Real mode) — 8086–80286

- Указатель: 32 бита = 16 бит (селектор сегмента) + 16 бит (смещение)
- Физический адрес = (сегмент × 16) + смещение
- Адресная шина: 20 бит → максимальный объём 1 Мбайт

Недостатки:

- Нет защиты памяти (можно обратиться к любому адресу)
- Программу нельзя перемещать в памяти

➤ Защищённый режим (Protected mode) — сегментная адресация

Селектор сегмента (16 бит) состоит из трёх полей:

Поле	Размер	Описание
Index	13 бит	Смещение в дескрипторной таблице (умножается на 8)
TI	1 бит	Выбор таблицы: 0 = GDT, 1 = LDT
RPL	2 бита	Запрашиваемый уровень привилегий

➤ Дескрипторные таблицы

Таблица	Назначение
GDT (Global Descriptor Table)	Глобальная таблица дескрипторов (для всех задач, ОС)
LDT (Local Descriptor Table)	Локальная таблица (для конкретной задачи)

Регистры:

GDTR — содержит базовый адрес и предел GDT

LDTR — содержит селектор LDT (сама LDT находится в GDT)

➤ Дескриптор сегмента (64 бита)

Поле	Описание
Базовый адрес	Начальный адрес сегмента в физической памяти (распределён по 2 байтам в начале и конце)
Предел (Limit)	Размер сегмента (распределён по 2 байтам)
TYPE	Тип сегмента (код, данные, стек, подчинённый код, только чтение, чтение/запись, только выполнение)
S	1 = сегмент памяти, 0 = системный объект
DPL	Уровень привилегий дескриптора (0–3)
G	Гранулярность: 0 = байты, 1 = страницы по 4 Кбайт
D	Разрядность: 0 = 16 бит, 1 = 32 бита (совместимость с 286)
P	Бит присутствия: 1 = сегмент в памяти, 0 = вызывает прерывание
A	Бит обращения (устанавливается доступе, используется для LRU)
U	Пользовательский бит (для ОС)
X	Зарезервировано (должен быть 0)

➤ **Регистр CR3**

- Содержит физический адрес каталога страниц (20 старших бит)
- Используется как точка входа для страничной трансляции

➤ **Ассоциативный буфер трансляции (TLB)**

Проблема: Сегментно-страничное преобразование требует до 3 обращений к памяти, что снижает производительность.

Решение: TLB (Translation Lookaside Buffer) — кэш для таблиц страниц, расположенный в процессоре.

Организация TLB (в процессорах Intel):

- 8 блоков по 4 строки (всего 32 записи)
- Разряды 12–14 линейного адреса → номер блока
- Внутри блока — ассоциативное сравнение поля тега (17 старших разрядов)

Формат строки TLB:

Поле	Описание
Тег	Старшие разряды линейного адреса
Физический адрес	Адрес начала страницы
Атрибуты	Аналогично полям в каталоге (P, R/W, U/S, A, D, PCD, PWT)
V	Бит достоверности

16. Виртуальная память. GDT, LDT, CR3

Виртуальная память — совокупность аппаратных и программных средств; нет возможности обратиться к любой ячейке (защищённый режим). У каждой задачи своя дескрипторная таблица → безопасно

➤ **Сегментная трансляция**

Логический адрес состоит из:

- Селектор сегмента (16 бит)
- Смещение (16 или 32 бита)

Селектор сегмента имеет структуру:

Поле Размер Описание

Index	13 бит	Индекс в дескрипторной таблице (умножается на 8)
TI	1 бит	0 = GDT, 1 = LDT
RPL	2 бита	Запрашиваемый уровень привилегий

✓ **GDT (Global Descriptor Table)**

GDT — глобальная дескрипторная таблица.

Характеристики:

- Одна на всю систему
- Содержит дескрипторы для всех задач (сегменты кода, данных, стека ОС)
- Хранит также дескриптор LDT
- Адрес и предел хранятся в регистре GDTR

Когда используется: TI = 0 в селекторе.

✓ **LDT (Local Descriptor Table)**

LDT — локальная дескрипторная таблица.

Характеристики:

- Своя для каждой задачи (процесса)
- Содержит дескрипторы сегментов конкретной задачи
- Дескриптор LDT находится в GDT

- Селектор LDT хранится в регистре LDTR

Когда используется: TI = 1 в селекторе.

Преимущество: изоляция процессов — каждая задача работает со своими сегментами.

✓ **CR3 (Control Register 3)**

CR3 — регистр управления, который содержит физический адрес каталога страниц.

Назначение:

- Точка входа для страничной трансляции
- Используется в защищённом режиме при включённой страничной адресации

Содержимое:

- Старшие 20 бит — физический адрес каталога страниц (выравнен на 4 КБ)
- Младшие биты — флаги (PCD, PWT)

Смена задачи: при переключении процессов ОС загружает в CR3 новый адрес каталога страниц.

➤ **Защита памяти**

Благодаря GDT, LDT и CR3 обеспечивается:

Механизм	Описание
Разделение GDT и LDT	Системные сегменты изолированы пользовательских
DPL (уровни привилегий)	0 (ядро) — самый высокий, 3 (пользователь) — самый низкий
CR3	У каждой задачи свой каталог страниц → изоляция памяти
Бит P	Отсутствующая страница вызывает прерывание для загрузки с диска

- GDT Глобальная таблица дескрипторов (системные сегменты, дескриптор LDT)
- LDT Локальная таблица дескрипторов (сегменты конкретной задачи)
- CR3 Указатель на каталог страниц (управление страничной трансляцией)

17. Управление памятью в 80386: R-режим, V-режим, P-режим

Процессор Intel 80386 получил возможность свободно переключаться между тремя режимами работы: R-режим (реальный), V-режим (виртуальный 8086) и P-режим (защищённый). Это позволило операционным системам получить «свободу маневра» и обеспечило обратную совместимость со старыми программами.

➤ **R-режим (Real mode — реальный режим)**

Реальный режим — режим работы процессора, совместимый с 8086/8088/80286.

Характеристики:

- Разрядность адреса: 20 бит
- Максимальный объём памяти: 1 Мбайт
- Физический адрес = (сегмент × 16) + смещение

- Нет защиты памяти — программа может обратиться к любому адресу
- Нет виртуальной памяти
- Используется при загрузке системы (DOS, начальная загрузка ОС)

Преимущества: Простота and Совместимость со старыми программами

Недостатки: Невозможность защитить ОС от пользовательских программ and Программу нельзя перемещать в памяти

➤ **V-режим (Virtual 8086 mode — режим виртуального 8086)**

Режим виртуального 8086 — режим, появившийся в 80386, позволяющий запускать старые программы, написанные для 8086, в защищённой среде под управлением ОС.

Характеристики:

- Эмулирует реальный режим для одной программы
- При этом операционная система сохраняет полный контроль над ситуацией
- Можно запускать несколько виртуальных 8086-машин одновременно
- Каждая V-задача изолирована от других

Как работает:

- Процессор переключается в V-режим из Р-режима
- Для V-режима используется страничная трансляция (через CR3)
- При прерываниях ОС получает управление

Преимущества:

- Совместимость со старыми программами
- Защита (старая программа не может повредить ОС или другие программы)
- Можно запускать несколько DOS-программ одновременно

4. Р-режим (Protected mode — защищённый режим)

Защищённый режим — основной режим работы современных операционных систем (Windows, Linux, macOS).

Характеристики:

- Разрядность: 32 бита (80386 и выше)
- Максимальный объём памяти: 4 Гбайта
- Сегментная адресация через GDT/LDT (с защитой)
- Страничная адресация через CR3, каталог и таблицы страниц
- Уровни привилегий (кольца защиты) — 4 уровня (0 – ядро, 3 – пользователь)
- Поддержка виртуальной памяти (выгрузка страниц на диск)

Режимы внутри Р-режима:

- Страничная трансляция может быть включена или выключена
- Если выключена — выход сегментации даёт физический адрес
- Если включена — выход сегментации даёт линейный адрес для страничной трансляции

R-режим	Начальная загрузка, DOS	Простота, совместимость с 8086
V-режим	Запуск старых DOS-программ в современных ОС	Совместимость + защита + многозадачность
Р-режим	Все современные ОС	Защита, виртуальная память, 32/64 бита

18. Защита памяти. Кольца защиты, шлюзы вызова, бит согласования.

Защита памяти — совокупность аппаратных и программных средств, обеспечивающих:

- Изоляцию пользователей друг от друга
- Защиту ОС от пользователей
- Контроль доступа к данным (чтение, запись, выполнение)
- Предотвращение повреждения программ из-за ошибок (переполнение стека)
- Защиту от намеренных попыток разрушить систему

➤ **Кольца защиты (уровни привилегий)**

В архитектуре Intel x86 используется 4 уровня привилегий (0 — самый привилегированный, 3 — самый низкий):

Уровень	Кому принадлежит
Кольцо 0	Ядро ОС (инициализация, управление памятью, защита)
Кольцо 1	Основная часть ОС (утилиты)
Кольцо 2	Служебные программы (драйверы, СУБД)
Кольцо 3	Прикладные программы пользователя

➤ **Где хранятся уровни привилегий**

Поле	Где хранится	Описание
DPL (Descriptor Privilege Level)	В дескрипторе сегмента	Допустимый уровень привилегий для доступа к сегменту
RPL (Requested Privilege Level)	В селекторе	Запрашиваемый уровень привилегий (2 младших бита)
CPL (Current Privilege Level)	В регистре CS	Текущий уровень привилегий выполняемой программы (равен DPL сегмента кода)

➤ **Правило доступа к сегменту**

Доступ к сегменту разрешается, если:

- **DPL ≥ max(CPL, RPL)**

- Численно больше = меньше привилегий (3 — самый низкий)
- Учитывается максимум из CPL и RPL (наименее привилегированный уровень)

- Это предотвращает атаку «троянский конь» (когда программа передаёт привилегированной процедуре чужой адрес)

Шлюз вызова (Call Gate)

Шлюз вызова — специальный дескриптор, обеспечивающий переход из менее привилегированного кольца в более привилегированное.

Структура шлюза вызова:

Поле	Описание
Селектор	Указывает на дескриптор сегмента кода вызываемой процедуры
Смещение	Точка входа в вызываемую процедуру
DPL	Уровень привилегий шлюза
P	Бит присутствия
Счетчик параметров	Сколько слов копировать из стека

Зачем нужен шлюз:

- Пользователь не знает физическое расположение привилегированной процедуры
- При изменении ОС достаточно обновить шлюз — программы пользователя не меняются
- Обеспечивается контроль доступа к привилегированным функциям

➤ Бит согласования (Conforming bit, бит подчинения C)

Бит согласования (C) находится в дескрипторе исполняемых сегментов (сегментов кода).

Особенности подчинённого сегмента (C = 1):

- Можно вызывать из менее привилегированного кольца без шлюза
- CPL не меняется — процедура выполняется с уровнем привилегий вызывающей программы
- Стеки не переключаются

Условие вызова: CPL вызывающей программы $\geq$ DPL подчинённого сегмента

Назначение: Подчинённый сегмент автоматически «подчиняется» кольцу вызывающей программы. Это подходит только для процедур, не содержащих привилегированных команд.

➤ Три случая вызова

Случай	Требования	Переключени е стеков
Одинаковые привилегии	Шлюз не обязателен	Нет
Вызывающая менее привилегированная	Требуется шлюз	Да (из TSS для целевого кольца)

Вызывающая более привилегированная	Запрещено (кроме подчинённых сегментов)	—
------------------------------------	---	---

19. Мультизадачность

Мультизадачность — способность процессора поддерживать выполнение нескольких задач (программ) одновременно, переключаясь между ними. Преимущество: пока одна программа ожидает завершения операции ввода-вывода (медленного внешнего устройства), процессор выполняет другую программу.

➤ Как Intel x86 поддерживает мультизадачность

Процессор поддерживает несколько задач путём переключения с одной задачи на другую. С каждой задачей ассоциируется сегмент памяти, содержащий всю информацию, необходимую для запуска и останова задачи. Этот сегмент называется TSS (Task State Segment) — сегмент состояния задачи.

➤ TSS (Task State Segment)

Основное назначение TSS — сохранять содержимое регистров задачи, когда процессор её не выполняет. Регистры — это разделяемый ресурс (используется всеми задачами), поэтому их состояние нужно сохранять при переключении.

Дескриптор TSS

TSS имеет специальный дескриптор в GDT. В байте доступа есть бит занятости (B):

Значение	Состояние
B = 1	Задача выполняется (занята)
B = 0	Задача не выполняется

Назначение бита B: в многопроцессорных системах предотвращает выполнение одной и той же задачи двумя процессорами одновременно.

Регистр задачи (TR)

TR (Task Register) — 16-битный регистр, содержащий селектор TSS в GDT для текущей выполняющейся задачи.

- Для ускорения доступа к TSS с регистром задачи ассоциируется теневой регистр, содержащий выбранный дескриптор
- Загрузить селектор в TR можно только привилегированной командой LTR

Переключение задач — специальной команды нет; через **CALL** (с возвратом) или **JMP** (без возврата), адресующие сегмент TSS.

- ❖ CALL помещает селектор вызывающей задачи в поле обратной связи вызываемой TSS, ставит NT=1 (флаг вложенной задачи); бит B вызывающей не меняет, в вызываемой ставит 1.
- ❖ JMP сбрасывает NT=0; сбрасывает B покидаемой задачи, ставит B=1 иницируемой.
- ❖ IRET при NT=1 переключает на задачу из поля обратной связи; при NT=0 — как обычно.

20. Система ввода-вывода

Система ввода/вывода (СВВ) — третий ключевой элемент архитектуры ЭВМ (наряду с процессором и памятью). Её назначение — обеспечить обмен информацией между ядром вычислительной машины и разнообразными внешними устройствами (ВУ).

➤ Две основные функции модуля ввода/вывода (МВВ)

Функция	Описание
«Большой» интерфейс	Обеспечение интерфейса с ЦП и памятью
«Малый» интерфейс	Обеспечение интерфейса с одним или несколькими периферийными устройствами

➤ Функции модуля ввода/вывода

Функция	Описание
Локализация данных	Возможность обращения к одному ВУ и адресации данных на нём
Управление и синхронизация	Координация перемещения данных между ЦП/памятью и ВУ
Обмен информацией	Передача данных между «большим» и «малым» интерфейсами
Буферизация данных	Компенсация разницы в скорости между ЦП и ВУ
Обнаружение ошибок	Контроль правильности передачи данных

➤ Способы организации адресного пространства ввода/вывода

Способ	Описание	Команды
Совмещённое (MMIO)	Порты ввода/вывода отображаются в адресное пространство памяти	Обычные команды (MOV и др.)
Выделенное (Port I/O)	Своё отдельное адресное пространство для портов	Специальные команды (IN, OUT)

21. Адресное пространство ввода-вывода

Адресное пространство ввода/вывода — это система адресации, позволяющая процессору выбирать конкретный модуль ввода/вывода или подключённое к нему внешнее устройство. Каждое устройство (или его порты) имеет свой уникальный адрес, по которому к нему обращается процессор.

➤ Два способа организации адресного пространства

Способ	Описание
Совмещённое адресное пространство (MMIO)	Порты ввода/вывода отображаются в адресное пространство памяти
Выделенное адресное пространство (Port I/O)	Порты имеют своё отдельное адресное пространство, не пересекающееся с памятью

❖ (MMIO)

Принцип: Для адресации модулей ввода/вывода отводится определённая область адресов основной памяти. Обращение к устройству выглядит как обращение к ячейке памяти.

Достоинства:

- Расширение набора команд — все команды работы с памятью применимы к портам
- Увеличение количества подключаемых устройств — не ограничено специальными командами
- Внепроцессорный обмен — возможен обмен между ВУ минуя ЦП (если есть команды пересылки память-память)
- Обмен с любым регистром — не только с аккумулятором

Недостатки:

- Сокращение области адресного пространства памяти — часть адресов занята под порты
- Усложнение декодирующих схем — нужно различать обращение к памяти и к порту
- Трудности распознавания операций ВВ среди других операций
- Сложность построения СВВ на простых модулях — сигналы управления не всегда могут координировать сложную процедуру ВВ

(Port I/O)

Принцип: Для обращения к модулям ввода/вывода применяются специальные команды и отдельная система адресов (своя нумерация, не пересекающаяся с памятью).

Достоинства:

- Адрес внешнего устройства в команде может быть коротким — количество портов ограничено
- Программы становятся более наглядными — операции ВВ выполняются специальными командами
- Разработка СВВ может проводиться отдельно от разработки памяти

Недостатки:

- Ввод/вывод производится только через аккумулятор — для передачи в другой регистр нужно 4 команды (сохранить AX, ввод, переслать, восстановить AX)
- Перед обработкой содержимое ВУ нужно переслать в ЦП — невозможен обмен между ВУ минуя процессор

22. Асинхронная работа ввода-вывода

Асинхронная работа ввода/вывода — это метод организации обмена данными между процессором и внешними устройствами, при котором процессор не ожидает завершения операции ввода/вывода, а продолжает выполнять другие задачи. Устройство сигнализирует о завершении операции с помощью прерывания.

➤ Синхронный vs асинхронный ввод/вывод

Харак-тика	Синхронный (программно-у-мый)	Асинхронный прерываниям)
Поведение ЦП	Ожидает завершения операции (циклический опрос)	Продолжает выполнение других задач
Оповещение завершении	ЦП сам обнаруживает по флагу готовности	Устройство посылает прерывание
Эфф-ность	Низкая (ЦП простаивает)	Высокая (ЦП занят полезной работой)
Сложность	Простая	Более сложная

23. Опрос флага готовности, прерывания, прямой доступ к памяти

Это три основных метода управления вводом/выводом, различающихся по степени участия центрального процессора в процессе обмена данными с внешними устройствами.

Метод	Участие ЦП	Эфф- вность	Сложность
Опрос флага готовности	Полное (активное ожидание)	Низкая	Простая
Прерывания	Только в начале и конце	Средняя	Средняя
Прямой доступ к памяти (ПДП)	Только при инициализации	Высокая	Высокая

Часть 1. Опрос флага готовности (программно-управляемый ввод/вывод)

➤ Принцип работы

ЦП сам активно опрашивает устройство, проверяя его готовность, и ожидает завершения операции.

Часть 2. Прерывания (ввод/вывод по прерываниям)

1. Принцип работы

ЦП запускает устройство и продолжает выполнять другие задачи. Устройство само сигнализирует прерыванием о завершении операции.

Часть 3. Прямой доступ к памяти (ПДП / DMA)

➤ Принцип работы

Обмен данными между внешним устройством и оперативной памятью происходит без участия ЦП. Управляет передачей контроллер ПДП (КПДП), который берёт на себя управление системной шиной.

➤ Почему нужен ПДП

Недостатки первых двух методов:

- Темп передачи ограничен скоростью, с которой ЦП может опросить/обслужить устройство
- ЦП вовлечён в каждую пересылку (через регистры AX, DX)
- При быстрых устройствах (диски) ЦП только и успевает, что обрабатывать прерывания

➤ Инициализация КПДП (4 параметра)

Параметр	Описание	Где хранится
Вид запроса	Чтение (ОП → ВУ) или запись (ВУ → ОП)	Логика управления
Адрес ВУ	Какое устройство участвует в обмене	Логика управления
Адрес начальной ячейки ОП	Откуда/куда передавать данные	Регистр адреса (РА)
Количество слов	Размер блока данных	Счётчик данных (СД)

24. Прерывания в реальном и виртуальном режиме.

Прерывание — основной способ общения всех компонентов компьютера.

Реальный режим. Два входа прерываний в 80286:

NMI — немаскируемые (не зависят от IF; ЦП не дожидается завершения команды — аварийные ситуации, например падение напряжения);

INTR — маскируемые (реакция зависит от IF: IF=0 запрещено, IF=1 разрешено). Первые пять типов — особые случаи, генерируемые ЦП, не зависят от IF.

Действия ЦП после вызова прерывания (одинаковы для всех видов):

1. ЦП завершает текущую команду.
2. Запоминает в стек текущее состояние (FLAGS, CS, IP).
3. В CS:IP помещается адрес из двойного слова, определённого типом прерывания.
4. Флаги IF и TF сбрасываются (прерывания запрещаются).
5. Возврат — командой IRET (извлекает FLAGS, CS, IP из стека).

Приоритеты прерываний. В 80286 приоритетов нет → подключается контроллер прерываний 8259A. Каждому устройству — уникальный приоритет; при одновременных прерываниях передаётся более приоритетное, второе запоминается. Конец обработки — особый код конца прерывания в порт контроллера. Прерывание прерывания: меньший приоритет → задержка; больший и IF=1 → текущий обработчик приостанавливается; больший и IF=0 → задержка. По умолчанию при входе в обработчик прерывания запрещены; разрешить — командой STI (правило хорошего тона для долгих обработчиков).

Виртуальный режим. Таблица прерываний содержит дескрипторы обработчиков — IDT (дескрипторная таблица прерываний); база и предел — в регистре IDT. Шлюзы в IDT:

шлюзы специальных прерываний — для особых случаев; при входе IF не меняется (чтобы особые случаи не выключали разделение времени, игнорируя прерывания таймера);

шлюзы задач — для внешних прерываний (инициируют вызов задачи, т.к. сигналы направляются в ОС). Обработка в зависимости от задачи: одна IDT для всех задач → нужен распределитель прерываний (через локальную таблицу задачи; должен быть в подчинённом сегменте) либо сам процессор 80286 как распределитель. Коды ошибок: многие особые случаи дают обработчику 16-битный код ошибки (включается в стек после адреса возврата; при шлюзе задачи адреса возврата нет — только код ошибки).

Виртуальный режим (V86 – Virtual 8086)

V86-режим — режим работы процессора, позволяющий запускать старые программы, написанные для реального режима (DOS), внутри защищённого режима под управлением современной ОС (Windows 9x/ME, OS/2).

25. Критика традиционной архитектуры фон Неймана.

Основные принципы архитектуры фон Неймана

Принцип	Описание
Двоичная система счисления	Переход от десятичной к двоичной, что позволяет использовать простые логические устройства
Программное управление	Работа ЭВМ контролируется набором команд, выполняемых последовательно друг за другом
Совместное хранение данных и программ	И данные, и команды хранятся в одной памяти, имеют одинаковый способ записи в двоичной системе
Адресация ячеек памяти	Все ячейки памяти имеют последовательно пронумерованные адреса
Условный переход	Возможность в ходе выполнения программы перейти к любому участку кода

Основные недостатки (критика)

1. «Семантический разрыв»

Определение: «Семантический разрыв» — это мера различия между принципами, лежащими в основе языков программирования высокого уровня, и принципами, определяющими архитектуру компьютера.

2. «Бутылочное горлышко» (фон-неймановское узкое место)

Определение: Термин, предложенный Джоном Бэкусом в 1977 году. Обозначает узкое место между операционным устройством (процессором) и памятью.

Суть проблемы:

- Скорость обработки информации в процессоре намного больше, чем скорость работы памяти
 - Память не успевает обеспечивать процессор новыми порциями информации
 - Процессор простаивает в ожидании данных
- Гарвардская архитектура как частичное решение

Принцип гарвардской архитектуры:

Память команд и память данных разделены
Каждая имеет свои шины адреса, данных и управления

Преимущества:

- Возможность одновременного доступа к командам и данным
- Интенсификация обмена между запоминающим устройством и процессором
- Повышение производительности

«Умная память» (Smart Memory)

Идея: Интегрировать запоминающие ячейки со схемами обработки данных непосредственно в память.

Преимущества:

Устранение «бутылочного горлышка»
Данные обрабатываются там, где хранятся

Актуальность критики сегодня

Проблема	Степень актуальности
Семантический разрыв	Сохраняется, но компиляторы становятся лучше
«Бутылочное горлышко»	Очень актуальна — разрыв в скорости между процессором и памятью растёт
Принципиальная последовательность	Актуальна, частично решается параллельными архитектурами

Парадокс: Несмотря на 80-летнюю критику, архитектура фон Неймана остаётся основой 99% современных компьютеров.